Neurocomputing 678 (2026) 133184

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Loc-Edit: Localized color editing in neural radiance fields

Haiyang Xu [a], Xia Xu [b,*], Yinghao Tan [a], Ziye Zhang [a], Nanhe Chen [c,d], Bin Pan [a], Zhenwei Shi [e]

[a] School of Statistics and Data Science, LEBPS, KLMDASR and LPMC, Nankai University, Tianjin, 300071, China
[b] School of Computer Science and Technology, Tiangong University, Tianjin, 300387, China
[c] Institute of Cyber-Systems and Control, Zhejiang University, Hangzhou, 310027, China
[d] Huzhou Institute of Zhejiang University, Huzhou, 313000, China
[e] Image Processing Center, School of Astronautics, Beihang University, Beijing, 100191, China

HIGHLIGHTS

- We propose Loc-Edit, a framework designed to enable convenient, flexible, and precise localized color editing for sampling-based NeRF variants.
- We introduce a novel ray-matching algorithm within Loc-Edit that identifies regions requiring editing directly in NeRF, without converting to other 3D representations, thereby avoiding accuracy loss.
- Experimental results show that Loc-Edit achieves more refined localized color editing compared with existing methods, while demonstrating superior operability and flexibility.

ARTICLE INFO

Communicated by H. Zhao

Keywords:
Neural radiance fields
3D scene editing
Neural rendering

ABSTRACT

Editing color in designated regions of neural radiance fields (NeRF) is particularly challenging, as NeRF is an implicit 3D representation that stores information in inoperable neural networks. Existing methods struggle to locate target regions and perform accurate editing since NeRF can only obtain depth and color from one view at a time through volumetric rendering. To address this limitation, we propose Localized Color Editing in Neural Radiance Fields (Loc-Edit), a method that enables refined localized color editing in designated regions of NeRF scenes by simply pasting image materials and directly painting. Loc-Edit preserves color details in additional plug-in networks, providing flexible editing approaches that allow different edits to be added and removed at will. To ensure 3D consistency within the scene, Loc-Edit proposes the ray-matching algorithm to identify the regions requiring editing in NeRF. Additionally, Loc-Edit enhances the volumetric rendering based on the distribution of sample points to improve rendering efficiency. Loc-Edit avoids the conversion of NeRF to other 3D representations through a novel ray-matching algorithm and plug-in network design, enabling convenient and precise color editing. Experimental results indicate that Loc-Edit achieves finer detail editing and exhibits better overall image quality than existing methods. The code and dataset are available at https://github.com/Lab-PANbin/Loc-Edit.

1. Introduction

The progress of Neural Radiance Fields (NeRF) [1–3] has significantly improved the quality of novel view synthesis, surpassing traditional methods such as point clouds [4,5], 4D light field functions [6,7], and multi-view stereo (MVS) [8,9]. Various types of improved NeRFs have been continuously proposed to address different challenges [10–15]. NeRF has also been extended to various other research fields, such as autonomous driving [16] and Simultaneous Localization And Mapping (SLAM) [17,18]. This underscores the importance of achieving controllable and localized color editing in NeRF. However, this task remains challenging due to the inherent characteristics of NeRF. On the one hand, NeRF is an implicit 3D representation that relies on volumetric rendering to generate images, which makes direct editing difficult. On the other hand, it encodes scene information within a parameter-sensitive neural network, where adjusting the parameters to achieve satisfactory editing is highly challenging.

A specific class of methods [19–23] is dedicated to globally editing the entire 3D scene to transform its style. These methods aim to obtain

* Corresponding author.
Email addresses: xuhaiyang@mail.nankai.edu.cn (H. Xu), xuxia@tiangong.edu.cn (X. Xu), yinghaotan@mail.nankai.edu.cn (Y. Tan), ziyezhang@mail.nankai.edu.cn (Z. Zhang), nanhe_chen@zju.edu.cn (N. Chen), panbin@nankai.edu.cn (B. Pan), shizhenwei@buaa.edu.cn (Z. Shi).

https://doi.org/10.1016/j.neucom.2026.133184
Received 20 November 2025; Received in revised form 11 February 2026; Accepted 26 February 2026
Available online 27 February 2026
0925-2312/© 2026 Published by Elsevier B.V.

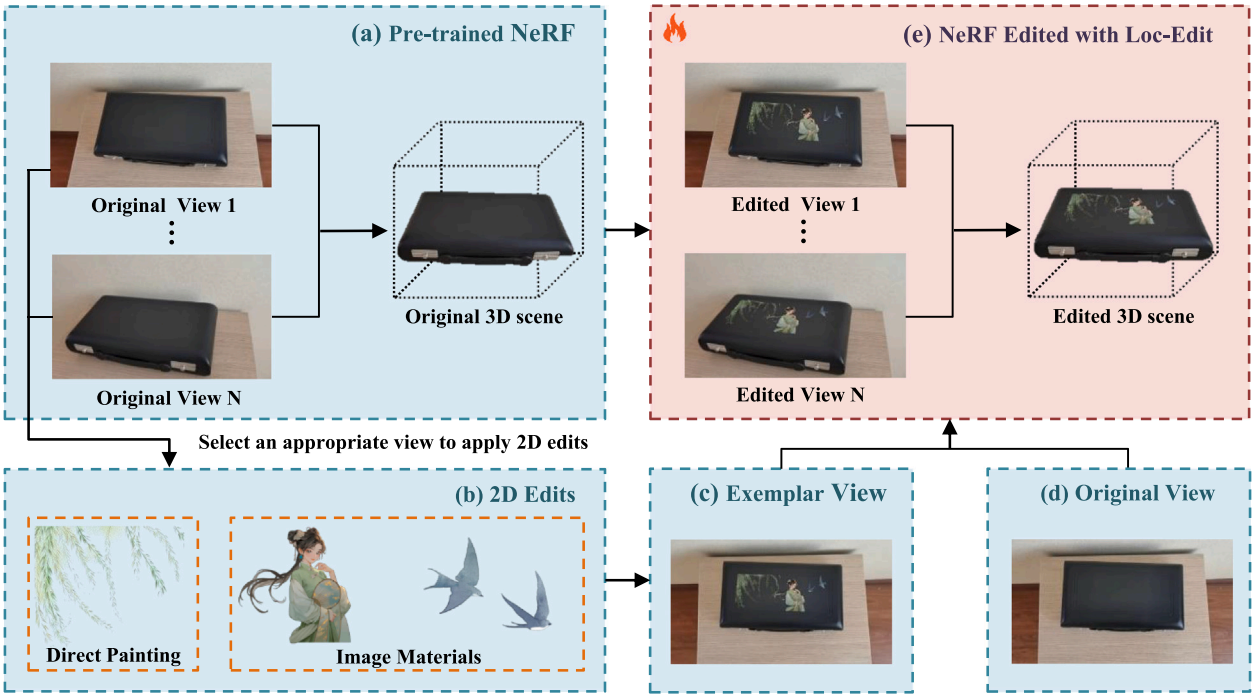


Fig. 1. Illustration of Loc-Edit workflow. Given (a) a pre-trained NeRF, users can edit the surfaces of objects by applying (b) 2D edits (pasting image materials and direct painting) at any position to create (c) an exemplar view. Given (c) and its corresponding (d) original view, the task of Loc-Edit is to produce (e) an edited NeRF that renders novel views while ensuring 3D consistency.

a stylized NeRF using content supervision from the original scene and style supervision from sources such as reference images. Other localized editing methods [24–28] aim for more controlled editing of specific semantic components in NeRF. Due to a lack of detailed supervision and prior information, these methods depend on large-scale pre-trained models like CLIP [29] and diffusion models [30–32] to introduce additional information.

However, stylization methods can only edit global colors and textures, but cannot achieve refined localized editing in specified regions. Feature-level methods, such as those using semantic correspondence, attempt to establish region-level matching between reference and target scenes. However, these methods rely on coarse semantic features that lack pixel-level precision, making them unable to accurately transfer fine details or handle complex geometric variations within matched regions. For localized editing methods, vague text prompts are often necessary to extract information from large-scale pre-trained models, which can result in inaccurate details. More importantly, when precise editing is required, particularly for editing specific scene regions using existing image materials, the above methods are not feasible.

To address these challenges, we propose Localized Color Editing (Loc-Edit) for Neural Radiance Fields. As illustrated in Fig. 1, given a pre-trained NeRF of the original scene, Loc-Edit enables users to select an appropriate view and apply 2D edits (pasting image materials or direct painting) to create an exemplar view. Using the exemplar view, its corresponding original view, and the camera poses of the original scene as input, Loc-Edit produces an edited NeRF that can render novel views with consistent localized edits across arbitrary viewpoints.

Specifically, Loc-Edit employs an additional plug-in network to learn the colors of the edited regions while keeping the pre-trained NeRF frozen. During rendering, Loc-Edit uses the ray-matching algorithm to identify the regions requiring editing in the 3D scene by estimating the positions of sample points. To enhance efficiency, volumetric rendering is adaptively optimized based on the sample weight distribution, enabling smooth interaction with the ray-matching algorithm. The plug-in network adopts a modular design that avoids retraining the full NeRF model while maintaining high flexibility. By operating directly within the NeRF representation rather than converting it to alternative 3D formats or manipulating deep features, Loc-Edit prevents unnecessary accuracy loss. As a result, the framework integrates naturally with sampling-based NeRF variants to support efficient and precise localized editing.

In summary, our contributions are as follows:

- We propose Loc-Edit, a framework designed to enable convenient, flexible, and precise localized color editing for sampling-based NeRF variants.
- We introduce a novel ray-matching algorithm within Loc-Edit that identifies regions requiring editing directly in NeRF, without converting to other 3D representations, thereby avoiding accuracy loss.
- Experimental results show that Loc-Edit achieves more refined localized color editing compared to existing methods, while demonstrating superior operability and flexibility.

The organization of the article is as follows: Section 2 reviews the related work on NeRF editing, Section 3 introduces the Loc-Edit method, Section 4 details the experimental setup and results, Section 5 discusses the findings and limitations, and Section 6 concludes the article.

2. Related work

2.1. Global stylization of NeRF

Some approaches aim to stylize NeRF [20,21,33] using a reference image. Rendered images from the stylized NeRF can closely resemble the reference image in terms of color and texture. They achieve the stylization of the entire 3D scene by aligning the radiance field with the reference image at the feature level. However, the inability of highly encoded features to provide precise supervision makes these methods incapable of localized and refined editing. On the other hand, alternative approaches employ diverse strategies for incorporating style guidance. NeRF-ART [22] achieves stylization by minimizing a designed cosine distance between the image CLIP [29] features of rendered images and

the text CLIP feature derived from a reference text. SNeRF [19] introduces a procedural approach utilizing a pre-trained 2D stylization network to obtain a stylized NeRF. These methods aim to leverage prior knowledge from pre-trained models. Nevertheless, they also involve extracting feature-level or image-level guidance from pre-trained models, which can be imprecise and arbitrary, resulting in a lack of detailed supervision.

2.2. Controllable editing of NeRF

There are plenty of methods attempting to achieve controlled and refined editing in NeRF. CLIP-NeRF [24] and Blending-NeRF [26] aim to edit NeRF by utilizing image and text encodings provided by CLIP. They enable tasks such as changing the color of a car by using pertinent text or a single image as the reference. FENeRF [34] and SketchFaceNeRF [35] employ GANs [36] to achieve controlled, 3D-consistent editing of NeRF for human faces. Edit-DiffNeRF [27] and Instruct-NeRF2NeRF [25] leverage the rapidly advancing diffusion models [37] to perform edits to objects spanning various categories in NeRF under the guidance of text prompts. Based on DFF [38] and DreamFusion [39], RePaint-NeRF [40] leverages both CLIP and diffusion models to edit the specified objects in NeRF. Concurrently, DE-NeRF [41] decouples intrinsic appearance and illumination to enable view-consistent appearance editing and high-frequency environmental relighting. Benefiting from the development of large-scale pre-trained models, NeRF has showcased a broad spectrum of capabilities in various controlled editing tasks. However, due to the limitations in the accuracy of information extracted from pre-trained models, pre-trained models are not adequately effective in guiding more detailed editing, such as pasting artistic pictures in 3D scenes. There are methods [42–46] that aim at directly editing NeRF. Nonetheless, it remains challenging to achieve convenient and refined editing while avoiding conversions to other 3D representations, which often lead to accuracy degradation.

2.3. Exemplar-based editing of NeRF

To address the ambiguity in supervision information, Ref-NPR [47] and SPIn-NeRF [48] attempt to provide more accurate and detailed supervision by using exemplar images. SPIn-NeRF achieves 3D-consistent scene inpainting by utilizing a 2D binary mask obtained from a 2D segmentation model [49] for precise guidance. Ref-NPR introduces a ray registration process using a styled exemplar image to acquire pseudo-ray supervision from new perspectives. Additionally, it leverages semantic correspondences from content images to perceptually fill occluded regions with a consistent style. However, these methods have their limitations. SPIn-NeRF lacks the capability for generation. Ref-NPR, despite utilizing exemplar images, still involves feature-level matching and alignment during editing, leading to uncontrollable and imprecise details.

3. Methodology

3.1. Preliminaries: neural radiance fields

NeRF encodes the color $\boldsymbol{c}$ and density $\boldsymbol{\sigma}$ of 3D sample points $\boldsymbol{x}$ from different view directions $\boldsymbol{d}$ using network f. Specifically, f represents a mapping function that translates the coordinates and view direction pair $(\boldsymbol{x}, \boldsymbol{d})$ into the corresponding color and density pair $(\boldsymbol{c}, \boldsymbol{\sigma})$. A ray is represented by $\boldsymbol{r}(t) = \boldsymbol{o} + t\boldsymbol{d}$, where $\boldsymbol{o}$ is the origin of the ray and t is the scalar representing the distance along the ray from $\boldsymbol{o}$ to a specific point $\boldsymbol{x}$. When predicting the color of a ray, N sample points are sampled along the ray, denoted as $\{\boldsymbol{x}_i = \boldsymbol{r}(t_i) | i = 1 \dots N, t_i < t_{i+1}\}$. The predicted color of the ray is determined through volumetric rendering:

$$\hat{C}(\boldsymbol{r}) = \sum_{i=1}^{N} T_i (1 - \exp(-\boldsymbol{\sigma}_i (t_{i+1} - t_i))) \boldsymbol{c}_i, \tag{1}$$

where $T_i = \exp\left(-\sum_{j=1}^{i-1} \boldsymbol{\sigma}_j (t_{j+1} - t_j)\right)$ represents transparency from t_1 to t_i, $\boldsymbol{\sigma}_i$ and $\boldsymbol{c}_i$ are the generated color and density by f at point $\boldsymbol{x}_i$, $w_i = T_i(1 - \exp(-\boldsymbol{\sigma}_i(t_{i+1} - t_i)))$ is the weight of the point $\boldsymbol{x}_i$. The loss function of the original NeRF is defined as:

$$L_{\text{rec}} = \sum_{\boldsymbol{r} \in R} \left\| C(\boldsymbol{r}) - \hat{C}(\boldsymbol{r}) \right\|^2, \tag{2}$$

where R represents all rays emitted from the pixels of the training images and $C(\boldsymbol{r})$ is the true color of the pixel.

3.2. Overall architecture

We present Loc-Edit, a method for refined localized color editing of a pre-trained NeRF by pasting image materials and directly painting, as illustrated in Fig. 2. Loc-Edit consists of training and rendering stages, respectively.

In the training stage, we first select a suitable view for easily pasting and painting. We denote the rendered image of the selected view as I_R. The exemplar image edited by pasting and painting from I_R is denoted as I_E. We first generate a binary mask M_r using I_E and I_R to identify the target rays corresponding to the 3D regions that require editing. We then employ an additional plug-in network g to learn the colors of these target rays with the help of the densities provided by the network f of the pre-trained NeRF. We generate a dictionary D_r to store the information of the target rays for the rendering stage.

During rendering, we introduce a ray-matching algorithm to determine whether each sample point belongs to the edited region by querying D_r. Sample points identified as edited are processed by the plug-in network g to generate their colors, while all other points are rendered by the original NeRF network f. To efficiently support this process, we adapt the volumetric rendering pipeline to work seamlessly with ray matching, thereby improving overall rendering efficiency.

3.3. Efficient targeted training

In order to achieve efficient training, the plug-in network g of Loc-Edit only trains for regions requiring editing in the 3D scene. Loc-Edit compares I_E to I_R to identify the edited regions. This results in M_r, where the pixel values determine whether the rays of the pixels should be included in training. Using the camera pose of I_R and M_r, the target rays associated with the edited regions and the corresponding pixel color labels provided by I_E are generated to train g, denoted as R_{train}. Because the target rays are generated from the same camera pose, the ray direction $\boldsymbol{d}$ does not offer valuable information; thus, g is only related to $\boldsymbol{x}$, independent of $\boldsymbol{d}$. Loc-Edit employs f to supply spatial density information during the training of g. We use L_{rec} as the loss function for training g. The volumetric rendering of a ray $\boldsymbol{r} \in R_{train}$ with direction $\boldsymbol{d}$ during training can be summarized as:

$$\boldsymbol{c}_i^g = g(\boldsymbol{x}_i),\ (\boldsymbol{c}_i^f, \boldsymbol{\sigma}_i^f) = f(\boldsymbol{x}_i, \boldsymbol{d}), \tag{3}$$

$$\hat{C}(\boldsymbol{r}) = \sum_{i=1}^{N} T_i (1 - \exp(-\boldsymbol{\sigma}_i^f (t_{i+1} - t_i))) \boldsymbol{c}_i^g. \tag{4}$$

After training, the necessary information of each target ray in R_{train} is stored in a dictionary D_r for the ray-matching algorithm to identify the edited regions during rendering. D_r stores the origin point $\boldsymbol{o}_r$, direction $\boldsymbol{d}_r$, and a slightly greater depth t_r compared to the point at which the ray intersects the object surface. The selection process of t_r of a ray $\boldsymbol{r} \in R_{train}$ can be summarized as:

$$m = \arg\min_{l} \left(\frac{\sum_{i=1}^{l} \boldsymbol{\sigma}_i^f}{\sum_{i=1}^{N} \boldsymbol{\sigma}_i^f} > \tau \right), \tag{5}$$

$$t_r = t_m, \tag{6}$$

where τ represents a small threshold. Loc-Edit only requires a small amount of training data and simplifies the complexity of network g, as g only needs to learn the colors in edited regions.

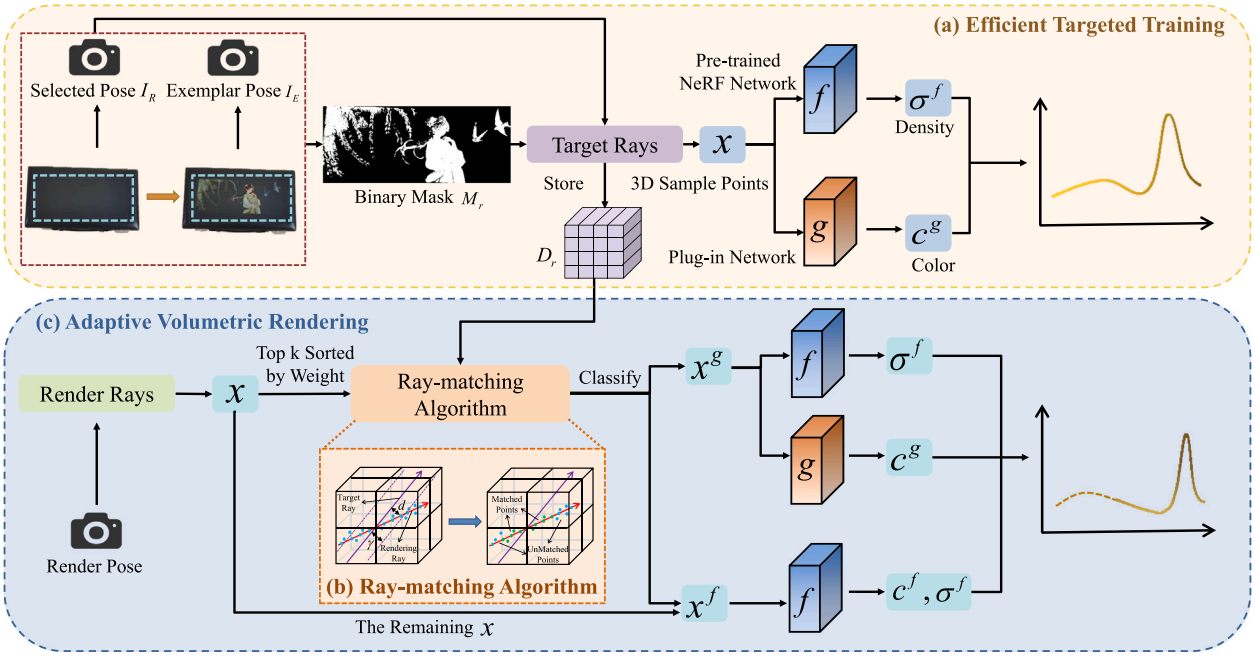


Fig. 2. Overview of Loc-Edit. The Loc-Edit is divided into two stages: (a) efficient targeted training and (c) adaptive volumetric rendering. The pre-trained network f is frozen during both stages, and the plug-in network g is frozen during rendering. (i) During training, we first select a pose for rendering the image I_R, which is then edited by pasting materials and painting to obtain the exemplar image I_E. Loc-Edit then generates a binary mask M_r to identify the target rays in the edited regions and trains g using the target rays. Loc-Edit stores the information of the target rays in D_r. (ii) During rendering, Loc-Edit proposes the (b) ray-matching algorithm to classify the sample points with large weights using D_r and then obtain predicted pixel colors.

3.4. Ray-matching algorithm

Since g only learns the colors of edited regions, we must determine whether each sample point belongs to these regions when rendering novel views. To achieve this, we introduce a two-step ray-matching algorithm, as shown in Fig. 3. First, we check whether a sample point lies within the spatial region covered by g, indicating that g has learned its color information. Second, we apply a depth constraint using the stored depth t_r in D_r to distinguish foreground points from background points, thereby preventing unintended background modification.

In the first step, we denote the distance from the sample point $\boldsymbol{x}$ to a target ray $\boldsymbol{r}$ in three-dimensional space by $d(\boldsymbol{x}, \boldsymbol{r})$. Using a sufficiently small threshold γ, if $d(\boldsymbol{x}, \boldsymbol{r})$ is less than γ, it indicates that point $\boldsymbol{x}$ is in close proximity to $\boldsymbol{r}$. If such a ray in D_r meeting the condition $d(\boldsymbol{x}, \boldsymbol{r}) < \gamma$ exists, we conclude that $\boldsymbol{x}$ belongs to the region covered by g, and therefore, g should be used to generate its color, rather than f. We evaluate each sample point with all rays in D_r to determine their respective regions. This evaluation forms a set of sample points S_1 defined as follows:

$$S_1 = \{\boldsymbol{x} \in R_{sample} \mid \exists \boldsymbol{r} \in D_r \text{ s.t. } d(\boldsymbol{x}, \boldsymbol{r}) < \gamma\}, \tag{7}$$

where R_{sample} is the set of all sample points during rendering.

The evaluation in the first step does not consider depth constraints, which could potentially affect the background portion of the image. In the second step, we introduce a depth constraint using the depth information t_r stored in D_r. For each sample point $\boldsymbol{x}$, we calculate its projection $t_{proj}(\boldsymbol{x}, \boldsymbol{r})$ onto each ray $\boldsymbol{r} \in D_r$. If $t_{proj}(\boldsymbol{x}, \boldsymbol{r})$ is less than the corresponding t_r of the ray, we conclude that $\boldsymbol{x}$ belongs to the foreground, and the background remains unaffected when g is used to generate its color. Since t_r only slightly exceeds the depth of the surface of the object intended for editing, g exclusively edits the color of the foreground. For the sample points in the empty space, their weights are extremely small according to Section 3.6, resulting in a negligible impact on the color of the rendered image. Therefore, we do not need to add additional restrictions to identify the points in the empty space. In the second step, we form another set S_2 of sample points defined as follows:

$$S_2 = \{\boldsymbol{x} \in R_{sample} \mid \exists \boldsymbol{r} \in D_r \quad \text{and its corresponding } t_r \text{ s.t. } t_{proj}(\boldsymbol{x}, \boldsymbol{r}) < t_r\}. \tag{8}$$

The sample points belonging to the set $S_1 \cap S_2$ are successfully matched points, and their colors are generated by g, while the colors of the remaining points are generated by f.

We propose the ray-matching algorithm for position estimation not only for its simplicity and directness, but also to ensure maximal accuracy in NeRF representation. Suppose we use methods like voxels to store the position information of the edited regions for matching. In that case, it not only involves the conversion of 3D representations but is also difficult to fully represent complex and rugged surfaces. If we use an implicit function to cover the edited regions, the design of this function is

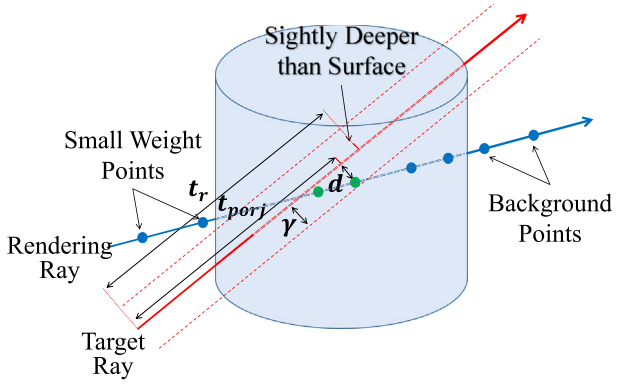


Fig. 3. The ray-matching algorithm is used for estimating the positions of sample points. The *green* points are successfully matched points, while the *blue* ones are not.

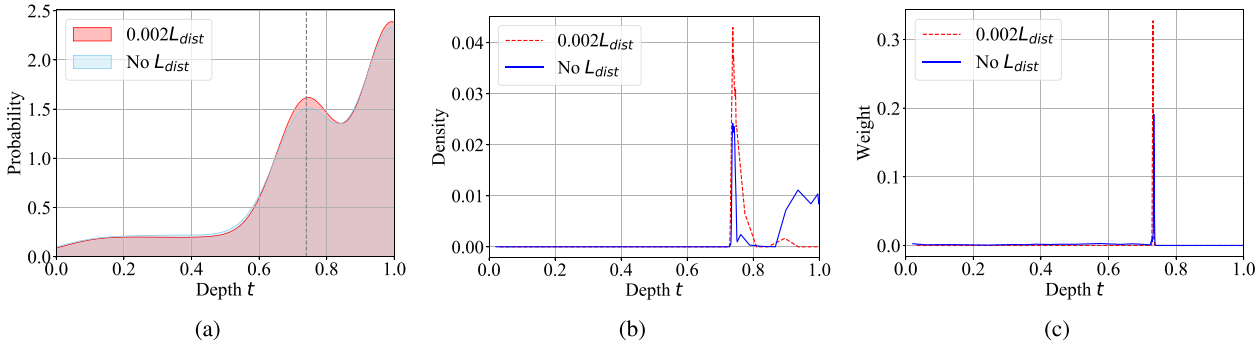


Fig. 4. Characteristics of sample points along a randomly selected ray. (a) Probability density of sample positions; the *gray dashed* line indicates the object surface depth. (b) and (c) show the changes in the density and weight of sample points along the ray depth t, respectively.

extremely challenging, as we may have to edit on any surface of any shape in the scene.

Let N denote the number of sample points per ray, k the number of top-weighted samples selected for matching, and M the number of rays stored in D_r. Computing sample weights has complexity $O(N)$, and selecting the top-k samples requires $O(NlogN)$. Ray matching is then performed only on these k samples, each compared against M stored rays, resulting in $O(kM)$. Therefore, the overall per-ray complexity of the proposed strategy is $O(NlogN + kM)$. In practice, k is significantly smaller than N, and M is limited to rays associated with edited regions, making the additional computational overhead manageable.

3.5. Adaptive volumetric rendering

Given the unimodal distribution of weights of the sample points discussed in Section 3.6, we do not need to engage in the inefficient process of matching for every sample point; instead, we focus on matching the most important points. Initially, the weights of all sample points $\{\boldsymbol{x}_1 \dots \boldsymbol{x}_N\}$ along the ray are computed using (1). Subsequently, these weights are sorted, and the ray-matching algorithm is conducted only for the sample points $\{\boldsymbol{x}_1 \dots \boldsymbol{x}_k\}$ with the highest top k weights. In cases where no sample points are matched to g, f is used to generate colors for all points along the ray. When there are sample points matched to g, their colors are generated by g, and the densities of other points along the ray are set to zero to prevent color mixing. The processing procedure of a ray with points matched to g can be summarized as:

$$\exists \{\boldsymbol{x}_1^g \dots \boldsymbol{x}_n^g\} \subseteq \{\boldsymbol{x}_1 \dots \boldsymbol{x}_k\},$$
$$\text{s.t.} \forall t = 1 \dots n \ \text{RM}(\boldsymbol{x}_t^g) \rightarrow g, \tag{9}$$
$$\forall \boldsymbol{x}_i \in \{\boldsymbol{x}_1^g \dots \boldsymbol{x}_n^g\}$$
$$\boldsymbol{c}_i^g = g(\boldsymbol{x}_i), (_, \boldsymbol{\sigma}_i^f) = f(\boldsymbol{x}_i, \boldsymbol{d}), \tag{10}$$
$$\forall \boldsymbol{x}_j \in \{\boldsymbol{x}_1 \dots \boldsymbol{x}_N\} \backslash \{\boldsymbol{x}_1^g \dots \boldsymbol{x}_n^g\},$$
$$(\boldsymbol{c}_j^f, _) = f(\boldsymbol{x}_j, \boldsymbol{d}) \ \text{set} \ \boldsymbol{\sigma}_j = 0, \tag{11}$$

where RM is the ray-matching algorithm. We then use all the sample points along the ray to predict the pixel color through volumetric rendering in Eq. (1). Setting the density $\boldsymbol{\sigma}_j$ of unmatched sampling points to 0 eliminates their influence on the final pixel color, thereby avoiding unnecessary color mixing between the edited content and the unedited content. Since the volume rendering weights are distributed in a unimodal manner, these unmatched points contribute very little on their own. Therefore, this operation does not introduce visible artifacts and ensures the clarity of the boundary of the edited region.

Through adaptive volumetric rendering, we process sample points differently depending on whether the rays contain matched points. This approach reduces the number of sample points involved in the ray matching algorithm, thereby saving computational resources and improving rendering efficiency. Moreover, given the unimodal distribution of sample point weights, the adaptive rendering selectively adjusts only the critical high-weight sample points within the edited regions, preserving rendering quality in the unedited regions.

3.6. Analysis of sample point distribution

In NeRF [1] and various NeRF variants [52,53], a hierarchical sampling approach is employed to generate a total of N samples. Initially, a stratified sampling approach is used to generate N_c coarse sample points during the coarse sampling phase. Then, additional N_f points are sampled based on the distribution of weights of coarse sample points during the fine sampling phase.

To better understand the characteristics of sample points, we randomly selected a ray for analysis. In Fig. 4(a), it is observed that a significant portion of the sample points along the ray tend to be located either near or at a greater depth from the surface of the object. Furthermore, as shown in Fig. 4(b), these sample points have a much higher density compared to those in empty space. Fig. 4(c) illustrates the variation of sample point weights with depth, which is characterized by a very steep unimodal distribution centered at the object surface. This phenomenon suggests that when predicting pixel color through volumetric rendering, the most significant impact comes from sample points close to the surface of objects, while the impact of other sample points is almost negligible.

To address the blurriness and collapsing issues in the depth estimation of NeRF, distortion loss [53] is proposed as a regularizer. The expression of the distortion loss is given as follows:

$$L_{\text{dist}} = \sum_{i,j} w_i w_j \left| \frac{t_i + t_{i+1}}{2} - \frac{t_j + t_{j+1}}{2} \right| + \frac{1}{3} \sum_i w_i^2 (t_{i+1} - t_i). \tag{12}$$

The objective of the distortion loss is to bring distant sample points into closer proximity during the fine sampling phase and simultaneously diminish the weights assigned to points in empty space while amplifying the weights of points situated in the vicinity of the object surfaces. This results in a more pronounced unimodal distribution in Fig. 4(c). As depicted in Fig. 5, the utilization of distortion loss allows rendering using only the subset of sample points with the highest weights, without substantial impact on the quality of the generated images.

4. Experiments

4.1. Implementation details

Loc-Edit is implemented based on the Mip-NeRF [52]. We set both N_f and N_c to 128 during sampling, and the coordinates of the sample points are limited between 0 and 1. To ensure the unimodal distribution of the weights of the sample points along the ray, the training loss of the pre-trained NeRF is set to L_{rec}+0.02L_{dist}. The pre-trained NeRF is trained for 200k iterations with a batch size of 2048. For the plug-in network g, we use an MLP with 8 layers and 256 hidden units, which uses ReLU



(a)



(b)

Fig. 5. Rendering quality with reduced sample counts. We use $L_{rec} + 0.002L_{dist}$ for training. There is no obvious difference between rendering with (a) 128 sample points and (b) 32 sample points.

internal activations and a sigmoid activation for color prediction in the final layer. We train g for 25k iterations with a batch size of 256. The training time for g is about 20 minutes. τ is set to 0.0005 when storing the depth information t_r of rays in the dictionary D_r. When rendering images, we set $\gamma = 0.002$ in the ray-matching algorithm and set $k = 32$ in adaptive volumetric rendering. We compare our Loc-Edit with three exemplar-based global stylization methods: Ref-NPR [47], SNeRF [19], and ARF [20], one localized editing method: NeuMesh [45] and two recent methods based on 3D Gaussian Splatting [54,55]: SGSST [50] and ReGS [51]. All experiments are conducted on a single NVIDIA RTX 3090 GPU (24GB VRAM).

Table 1
Quantitative comparisons with global stylization methods on the real-world dataset.

Method	MP			LPIPS	Robustness
	diff	sim	MP		
ARF [20]	0.233	0.626	0.48	0.62	20.91
SNeRF [19]	0.187	0.640	0.52	0.59	29.96
Ref-NPR [47]	0.197	0.622	0.50	0.61	21.21
SGSST [50]	0.255	0.676	0.50	0.50	27.74
ReGS [51]	0.199	0.691	0.55	**0.46**	36.43
Loc-Edit (Ours)	**0.043**	**0.865**	**0.83**	0.48	**43.59**

4.2. Dataset

In our experiments, we evaluate our method on a novel real-world dataset, which contains 12 scenes (7 scenes are captured by a high-resolution camera, and 5 scenes are obtained from 720P videos). For each scene, we edit the rendered image of a selected pose by pasting image materials and directly painting. We have edited various object surfaces, including uneven surfaces like stone tablets, flat surfaces like tables, and curved surfaces like cups. All the data is in the form of LLFF [6]. Camera poses are estimated using COLMAP [56]. Meanwhile, we also utilized four scenarios from the DTU dataset [57], namely dtu_scan40, dtu_scan55, dtu_scan65 and dtu_scan105.

4.3. Comparison with global stylization methods

In this paper, we propose a general framework for fine-grained localized color editing that can be applied to all sampling-based NeRF variants. However, to the best of our knowledge, no similar framework has been proposed so far. Global stylization is usually applicable to multiple NeRF variants, but it is not designed for fine-grained editing. Localized editing methods usually propose a new NeRF for editing and cannot be applied to other NeRF variants.

4.3.1. Quantitative comparison

We adopt three different metrics to compare Loc-Edit with comparison methods quantitatively. Inspired by Li et al. [58], we calculate manipulative precision (MP) from edited novel views and corresponding unedited views rendered by base models. Following [58], MP is defined as

$$\mathrm{MP} = (1 - \mathrm{diff}) \times \mathrm{sim}, \tag{13}$$

where diff is L_1 pixel distance between the images of novel views rendered by the edited NeRF and pre-trained base NeRF, and sim is the cosine similarity calculated using feature vectors of the exemplar image and rendered edited images extracted by pre-trained CLIP encoders. Additionally, we use the LPIPS [59] to measure the consistency between the exemplar image and images of novel views rendered by the edited NeRF. Following [47], we train the models in an iterative way to evaluate the robustness of the methods. We first train a model using the original exemplar image. In the following iterations, we use the model from the previous iteration to render an image from another view as an exemplar to train a new model. By training models repetitively, we obtain groups of rendered images from distinct views rendered by models in different iterations. We evaluate the robustness of the methods by calculating the average PSNR of images from the same view and in different iterations. The robustness metric measures whether a method can maintain stable and consistent results when the reference images are imperfect or vary across iterations, which commonly occurs in real-world applications. The average PSNR indicates the consistency of the rendered images, which measures the stability of the methods. Quantitative results, including diff and sim that are used when calculating MP, are shown in Table 1.

The results demonstrate that Loc-Edit achieves higher edited region fidelity, background consistency, and perceptual similarity. Note that ReGS achieves the lowest LPIPS. This is mainly because its Gaussian-domain optimization emphasizes perceptual or style similarity, which tends to reduce LPIPS. However, this often comes with stronger global style shifts and texture hallucination, weakening local controllability and background faithfulness. Consequently, MP and robustness are inferior to our method, which focuses on fine-grained, region-specific editing while preserving unedited areas.

4.3.2. Qualitative comparison

We present some of our experimental results in Fig. 6. We edit the uneven surface of the stone tablet in scene (a) by painting directly, and edit the curved surface of the cello box in scene (b) by pasting the image material. In these two scenes, the rendered images from the edited NeRF of Loc-Edit demonstrate significantly better overall quality than all comparison methods. The rendered images of Ref-NPR [47], SNeRF [19], ARF [20], SGSST [50], and ReGS [51] tend to be blurry or lack fine-grained control in the edited regions. In contrast, Loc-Edit can perform precise editing to maintain the details of 2D edited regions as it does not

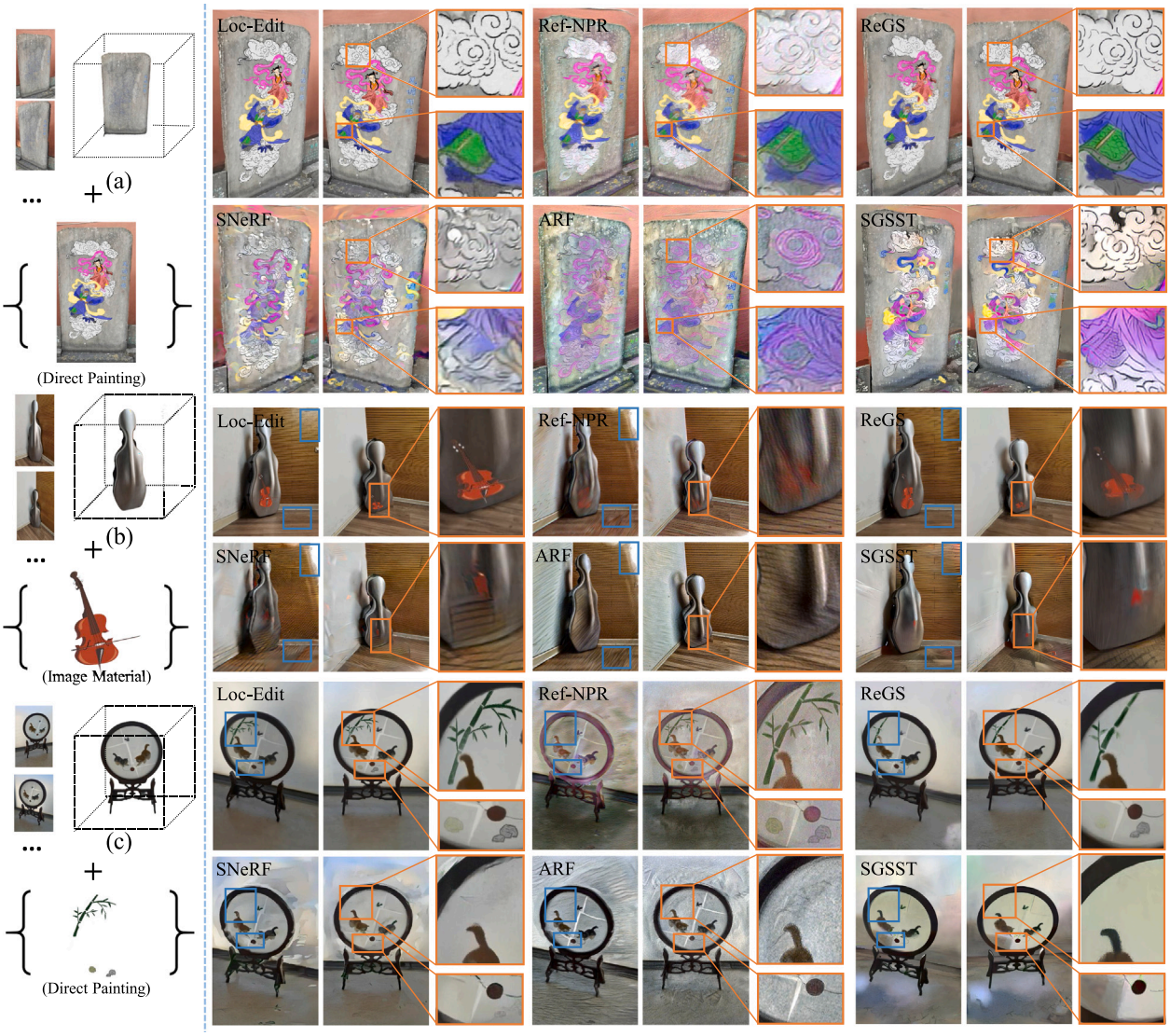


Fig. 6. Qualitative comparisons. Loc-Edit produces finer editing details within the edited regions marked in the *orange* boxes and does not affect the unedited regions marked in the *blue* boxes.

involve any feature-level manipulation. Loc-Edit can precisely identify the regions that need to be edited through the ray-matching algorithm, without affecting the unchanged regions, while contrast methods cause blurring and distortion in the background.

4.4. Comparison with localized editing methods on DTU

In order to further demonstrate the editing quality of Loc-Edit on DTU, we compare three methods: NeuMesh [45] (mesh-based localized editing), ReGS [51] (Gaussian-based method that supports both stylization and editing), and our Loc-Edit. NeuMesh is a mesh-based implicit field that can be used for color and texture editing, presenting a mesh representation by encoding disentangled geometry and texture codes on mesh vertices. However, NeuMesh can only edit its specific mesh-based representation and heavily relies on the mesh for editing tasks, which may lose some accuracy of the neural radiance field. ReGS provides an alternative localized editing route in the Gaussian domain; we adapt its official localized editing configuration for DTU.

As shown in Table 2, Loc-Edit achieves the highest MP and sim scores, demonstrating superior editing fidelity and perceptual consistency, while NeuMesh attains the lowest diff due to its mesh-constrained editing. ReGS achieves the best LPIPS, benefiting from its Gaussian-domain perceptual optimization, yet exhibits lower MP, indicating that perceptual similarity alone does not guarantee fine-grained local controllability.

Table 2
Quantitative comparisons with localized editing methods on the DTU dataset.

Method	diff	sim	MP	LPIPS
NeuMesh [45]	**0.046**	0.891	0.85	0.74
ReGS [51]	0.380	0.710	0.44	**0.65**
Loc-Edit (Ours)	0.055	**0.919**	**0.87**	0.68

The qualitative comparison is shown in Fig. 7. In all scenes, Loc-Edit, NeuMesh and ReGS all maintain 3D consistency. When dealing with lines and edges, Loc-Edit shows better results. For example, in scene (a), the red crown pattern on the skull is contaminated to nearly black. When editing the colors of other patterns, the colors edited by NeuMesh have changed significantly, and some are even difficult to recognize. Moreover, ReGS introduces noticeable distortions in the edited textures. In scene (b), at the edge of the painting, the edited lines of NeuMesh are distorted and deformed, while both Loc-Edit and ReGS

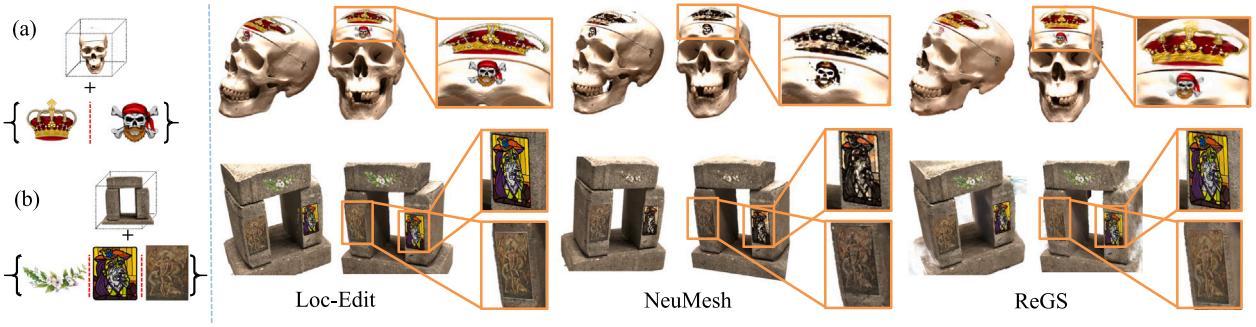


Fig. 7. Loc-Edit surpasses mesh-based NeuMesh and 3DGS-based ReGS in editing quality on (a) dtu_scan65 and (b) dtu_scan40.

maintain the straight edges well. The colors edited by Loc-Edit are not distorted because Loc-Edit directly changes the color values of the sampling points. However, NeuMesh has serious color distortion when editing complex patterns. ReGS, although effective in the Gaussian domain, sometimes introduces blur or loses fine high-frequency details on DTU under localized edits compared with Loc-Edit.

Loc-Edit, which adopts a NeRF-based representation, differs fundamentally from 3DGS-based methods such as SGSST and ReGS in terms of both modeling paradigm and editing mechanism. NeRF-based methods implicitly represent 3D scenes as continuous volumetric radiance fields and rely on volumetric rendering to accumulate color and density along rays, which naturally facilitates the handling of occlusions and spatial continuity. In contrast, 3DGS-based methods use explicit Gaussian primitives as the basic representation units, where color and opacity are directly associated with discrete points, making their editing performance dependent to some extent on the distribution density and coverage of the Gaussian representations. As a result of these differences, the two categories of methods exhibit distinct characteristics in editing tasks: NeRF-based approaches are better suited for consistency control and density-aware editing through volumetric rendering, whereas 3DGS-based methods can be more efficient for local attribute modifications but are more susceptible to representation discreteness in fine-grained editing, particularly in regions with complex occlusions or boundaries.

4.5. Effects of hyperparameters

Loc-Edit involves several hyperparameters, among which γ and k play a central role in determining the behavior of the editing process. In this subsection, we analyze the effects of these two hyperparameters on the edited results, as they directly influence the spatial selection of sample points and the visual quality of the edited regions.

Loc-Edit needs to select an appropriate hyperparameter γ for the ray-matching algorithm, where γ determines which sample points are included in the set S_1. As illustrated in Fig. 8, varying γ leads to different editing behaviors. A larger value of γ includes more sample points near object boundaries into S_1, which may introduce blurring and contamination in the edge regions of the edited areas. In contrast, when γ is set too small, sample points inside the target regions may be incorrectly excluded from S_1, resulting in incomplete editing in the rendered image.

In adaptive volumetric rendering, k denotes the number of sample points along each ray that are selected by the ray-matching algorithm. As discussed in Section 3.6, an excessively large k is unnecessary and leads to wasted computational resources. However, a too-small k may also degrade the visual quality of the edited regions, as shown in Fig. 9. This is because sample points are not strictly concentrated on the object surface, but are distributed both on the surface and within a depth range beyond it. When k is insufficient to include all surface-related sample points along a ray, the resulting pixel color may be adversely affected.

We emphasize that γ and k are the most critical hyperparameters in Loc-Edit, as they directly control the spatial selection of sample points during editing and the rendering quality of the edited regions. Other hyperparameters mainly serve implementation and numerical stability purposes. Specifically, the parameter τ is only used to determine the depth information stored in the ray dictionary D_r and does not participate in the ray-matching process or affect the construction of S_1. As analyzed in Section 3.6, the sample point weights exhibit a sharply peaked unimodal distribution concentrated near object surfaces, making the influence of τ on the final editing results negligible.

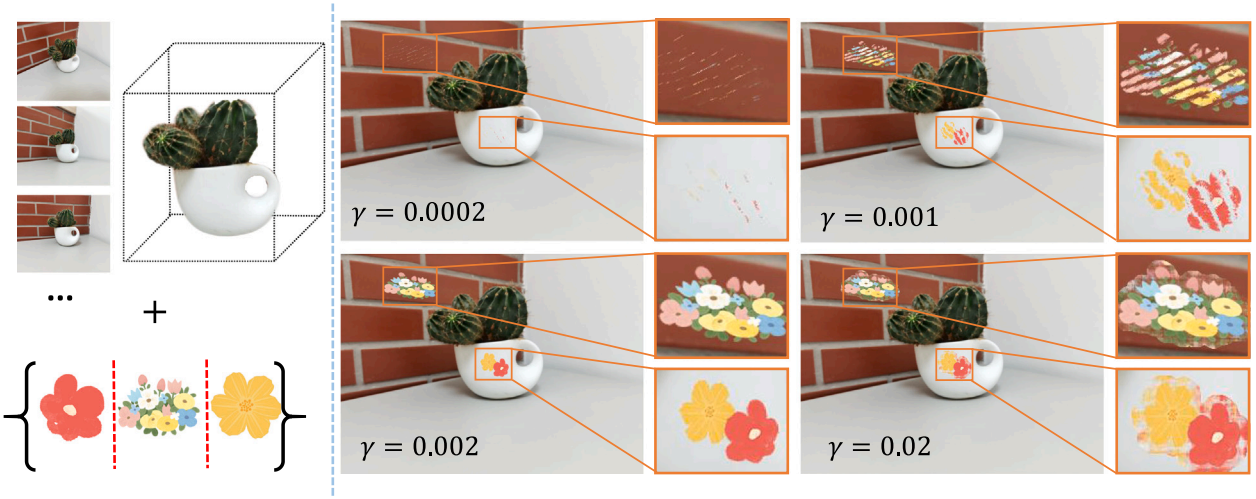


Fig. 8. Influence of γ on matching performance. We test the threshold γ on both the flat wall surface and the curved vase surface.

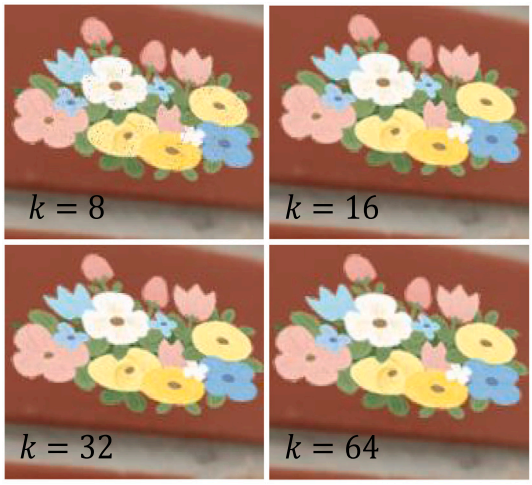


Fig. 9. Test on the hyperparameter k.



Fig. 10. Examples of editing with Loc-Edit. (a) A 3D scene of an apple and an orange, where imperfections on the fruit surfaces are removed. (b) A bottle, where the text color on its surface is altered.

Similarly, the plug-in network g adopts a standard NeRF-style MLP architecture and is responsible only for color regression within the edited regions, without determining their spatial extent. We observe stable performance under different network configurations. Furthermore, we reconstruct scenes using images of different resolutions and apply the same set of hyperparameters during editing, consistently obtaining refined results. These observations indicate that Loc-Edit is not sensitive to hyperparameter choices in practice.

4.6. Operability and flexibility

Loc-Edit offers high operability and flexibility, not only because it can edit NeRF by simply publishing image materials and drawing directly, but also because it allows for the arbitrary combination and deletion of edits from different views. We demonstrate the flexibility and operability of Loc-Edit through two different editing approaches and use some examples to better demonstrate the application potential of Loc-Edit.

The first flexible editing approach, as shown in Fig. 11, is to edit two surfaces in the scene using two different views. We can use one plug-in network to obtain the results of multiple edits from multiple views. For areas that are difficult to edit from a certain viewpoint, we can switch the viewpoint to facilitate editing while avoiding the distortion and deformation of the materials that may be caused by a tilted viewpoint.

Furthermore, benefiting from the plugin design of the network of Loc-Edit, As shown in Fig. 12, we can train multiple plugin networks for different edits in the same scene and combine these networks arbitrarily for editing. Loc-Edit can easily delete or add edits in different regions. Since different edits have independent plug-in networks and D_r for storing information of the rays, different edits will not interfere with each other even if they are displayed on the same object surface closely, and there is no impact on the refinement of the editing.

As shown in Fig. 10, our method has many application scenarios and can meet various needs for color editing. In scene (a), we remove the defects of the fruits through editing. In scene (b), we change the color of the writing on the distorted surface of the vase.

5. Discussions

5.1. Scalability and rendering efficiency

The primary objective of this paper is to validate the feasibility of a new paradigm for precise NeRF editing through direct painting and image-based material transfer. Therefore, our focus is on verifying whether accurate and controllable editing can be achieved in 3D scenes. Consequently, experiments on large-scale scenes are not included in this study, which we acknowledge as a limitation of the current work.

The adaptive sampling strategy in Loc-Edit improves rendering efficiency during the editing process and is sufficient for the small- to medium-scale edited scenes considered in this paper, which represent common real-world use cases. Rendering large-scale scenes with NeRF still involves inherent structural bottlenecks that cannot be fully eliminated by a single editing method. Since editing is typically performed after the scene has been fully pretrained, this paper focuses on keeping the additional computational overhead introduced during editing manageable. For larger or more complex scenes, several optimization directions can be explored. The ray-matching process can be accelerated using spatial indexing structures (e.g., octrees) to reduce matching

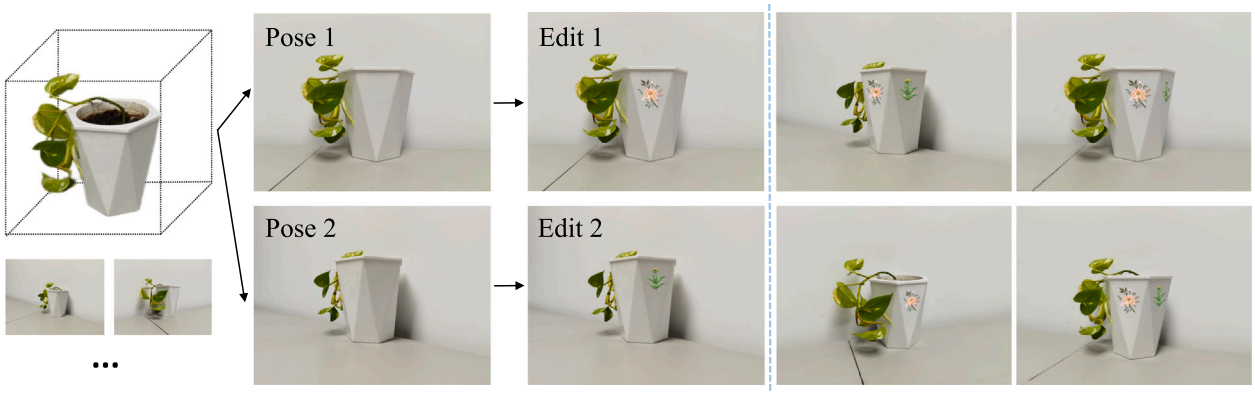


Fig. 11. Multi-material editing with a single plug-in network. We select two viewpoints to paste different image materials and train one plug-in network to obtain the edited NeRF. The four images on the *right* show novel views rendered from the edited NeRF.

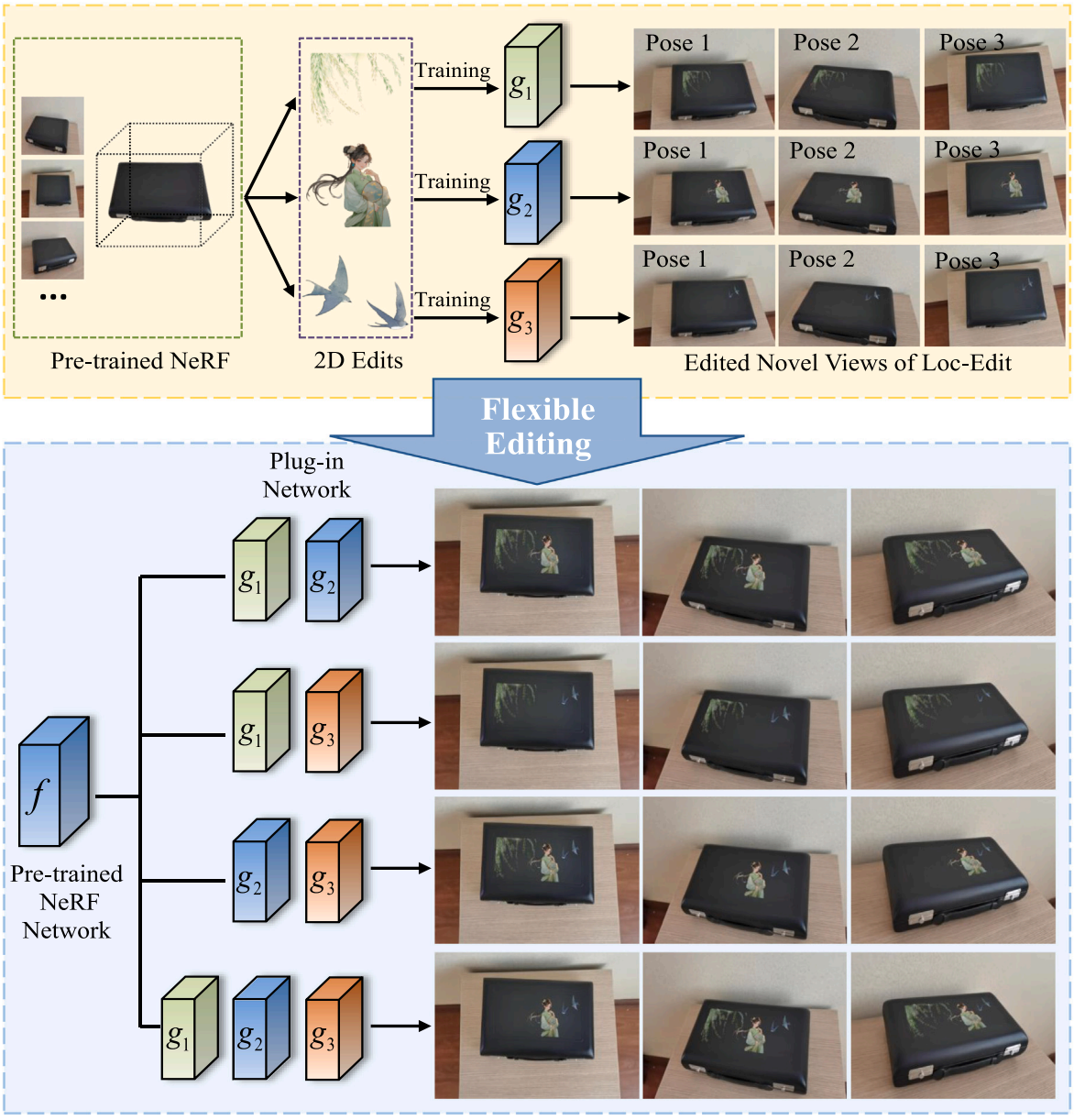


Fig. 12. Loc-Edit can freely combine different plug-in networks for editing.

complexity, and further improved through GPU-parallel execution, as matching operations across rays are independent. In addition, the plug-in network can be optimized via lightweight architectural design and model compression, since it only learns color distributions in edited regions. Finally, sparse sampling strategies can be applied by reducing sampling density or terminating matching early for rays that do not intersect edited regions, providing additional scalability improvements. In large-scale scenes where edited areas typically occupy only a small portion of the entire scene, such strategies can significantly reduce unnecessary computation and improve scalability.

5.2. Transferability across scenes

The plug-in network in Loc-Edit is scene-specific and cannot be directly reused across different NeRF scenes, as it learns color mappings defined within the 3D coordinate system and geometric structure of a particular scene. Differences in spatial range, surface distribution, and density fields prevent straightforward cross-scene reuse. Nevertheless, several potential directions for transfer learning can be considered. When scenes share similar surface geometries, knowledge transfer may be enabled through coordinate alignment and subsequent fine-tuning. Introducing a canonical or normalized coordinate space may further reduce dependency on scene-specific spatial ranges and improve transfer stability. It is also possible to decompose scene-agnostic and scene-specific components within the plug-in network, for example by reusing early layers that capture general color-editing patterns while adapting later layers to geometry-dependent features. These directions suggest that although direct reuse is not currently supported, the proposed framework provides a foundation for future cross-scene transfer learning.

6. Conclusion

In this paper, we have presented Loc-Edit, a method for convenient and precise localized color editing in NeRF by simply pasting image materials and directly painting. Loc-Edit employs a plug-in network for edits in designated regions. We propose the ray-matching algorithm to identify the edited regions and propose adaptive volumetric rendering

to enhance rendering efficiency. The editing process of Loc-Edit does not involve any conversion of 3D representations that may cause accuracy loss. Experimental results have shown that Loc-Edit achieves higher editing quality on the constructed dataset than comparison methods. Additionally, we have analyzed the potential effects of hyperparameters and discussed the operability and flexibility of Loc-Edit. Although our method still needs improvement in rendering efficiency, we believe that it has great potential for application. This paper may exhibit limited performance in large-scale scenes. However, strategies such as spatial data structures, GPU-parallel matching, lightweight design of the plug-in network, and sparse sampling may enhance rendering efficiency and scalability.

CRediT authorship contribution statement

Haiyang Xu: Writing – original draft, Methodology. **Xia Xu:** Writing – review & editing, Conceptualization. **Yinghao Tan:** Visualization, Validation. **Ziye Zhang:** Software, Data curation. **Nanhe Chen:** Visualization, Validation. **Bin Pan:** Methodology, Funding acquisition. **Zhenwei Shi:** Funding acquisition, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Grammarly in order to check and correct grammatical errors in the English writing. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The work was supported by the National Key Research and Development Program of China under Grant 2022YFA1003800, the National Natural Science Foundation of China under the Grant 62571273, Grant 62125102 and Grant U24B20177.

Data availability

Data will be made available on request.

References

[1] B. Mildenhall, P.P. Srinivasan, M. Tancik, J.T. Barron, R. Ramamoorthi, R. Ng, NeRF: representing scenes as neural radiance fields for view synthesis, Commun. ACM 65 (2021) 99–106, https://doi.org/10.1145/3503250
[2] K. Gao, Y. Gao, H. He, D. Lu, L. Xu, J. Li, NeRF: Neural radiance field in 3D vision, a comprehensive review, arXiv preprint arXiv:2210.00379, 2022.
[3] W. Zhang, R. Xing, Y. Zeng, Y.-S. Liu, K. Shi, Z. Han, Fast learning radiance fields by shooting much fewer rays, IEEE Trans. Image Process. 32 (2023) 2703–2718.
[4] R. Fu, Q. Li, C. Wen, N. An, F. Tang, A novel framework for learning bézier decomposition from 3D point clouds, IEEE Trans. Circuits Syst. Video Technol. (2024), https://doi.org/10.1109/TCSVT.2024.3514740
[5] Y. Jiang, H. Zhang, L. Li, D. Liu, Z. Li, Sparse point clouds assisted learned image compression, IEEE Trans. Circuits Syst. Video Technol. (2024), https://doi.org/10.1109/TCSVT.2024.3522621
[6] B. Mildenhall, P.P. Srinivasan, R. Ortiz-Cayon, N.K. Kalantari, R. Ramamoorthi, R. Ng, A. Kar, Local light field fusion: practical view synthesis with prescriptive sampling guidelines, ACM Trans. Graph. 38 (2019) 1–14.
[7] Z. Qu, Z. Zou, V. Boominathan, P. Chakravarthula, A. Pediredla, Event fields: capturing light fields at high speed, resolution, and dynamic range, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 26910–26920.
[8] J. Li, Z. Lu, Y. Wang, J. Xiao, Y. Wang, NR-MVSNet: learning multi-view stereo based on normal consistency and depth refinement, IEEE Trans. Image Process. 32 (2023) 2649–2662.
[9] Y. Furukawa, J. Ponce, Accurate, dense, and robust multiview stereopsis, IEEE Trans. Pattern Anal. Mach. Intell. 32 (2009) 1362–1376.
[10] T. Zhang, L. Zhang, F. Zhang, S. Zhao, Y. Zhou, I-DACS: always maintaining consistency between poses and the field for radiance field construction without pose prior, IEEE Trans. Circuits Syst. Video Technol. (2024), https://doi.org/10.1109/TCSVT.2024.3492315
[11] K. Zhang, G. Riegler, N. Snavely, V. Koltun, NeRF++: analyzing and improving neural radiance fields, arXiv preprint arXiv:2010.07492, 2020.
[12] S. Li, Z. Xia, Q. Zhao, Representing boundary-ambiguous scene online with scale-encoded cascaded grids and radiance field deblurring, IEEE Trans. Circuits Syst. Video Technol. 34 (2024) 2026–2040, https://doi.org/10.1109/TCSVT.2023.3300170
[13] M. Chen, L. Wang, Y. Lei, Z. Dong, Y. Guo, Learning spherical radiance field for efficient 360° unbounded novel view synthesis, IEEE Trans. Image Process. 33 (2024) 3722–3734.
[14] S. Guo, Q. Wang, Y. Gao, R. Xie, L. Li, F. Zhu, L. Song, Depth-guided robust point cloud fusion NeRF for sparse input views, IEEE Trans. Circuits Syst. Video Technol. 34 (2024) 8093–8106, https://doi.org/10.1109/TCSVT.2024.3385360
[15] X. Liu, H. Xue, K. Luo, P. Tan, L. Yi, Genn2n: generative nerf2nerf translation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 5105–5114.
[16] Z. Wu, T. Liu, L. Luo, Z. Zhong, J. Chen, H. Xiao, C. Hou, H. Lou, Y. Chen, R. Yang, et al., MARS: an instance-aware, modular and realistic simulator for autonomous driving, in: Proceedings of the CAAI International Conference on Artificial Intelligence, 2023, pp. 3–15.
[17] M.M. Johari, C. Carta, F. Fleuret, Eslam: efficient dense slam system based on hybrid representation of signed distance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 17408–17419.
[18] E. Sandström, Y. Li, L. Van Gool, M.R. Oswald, Point-slam: dense neural point cloud-based slam, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 18433–18444.
[19] T. Nguyen-Phuoc, F. Liu, L. Xiao, SNeRF: stylized neural implicit representations for 3D scenes, ACM Trans. Graph. 41 (2022) 1–11, https://doi.org/10.1145/3528223.3530101
[20] K. Zhang, N. Kolkin, S. Bi, F. Luan, Z. Xu, E. Shechtman, N. Snavely, ARF: artistic radiance fields, in: Proceedings of the European Conference on Computer Vision (ECCV), 2022, pp. 717–733.
[21] K. Liu, F. Zhan, Y. Chen, J. Zhang, Y. Yu, A. El Saddik, S. Lu, E.P. Xing, StyleRF: zero-shot 3D style transfer of neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 8338–8348.
[22] C. Wang, R. Jiang, M. Chai, M. He, D. Chen, J. Liao, NeRF-Art: text-driven neural radiance fields stylization, IEEE Trans. Vis. Comput. Graph. (2023), https://doi.org/10.1109/TVCG.2023.3236695
[23] J.-W. Liu, Y.-P. Cao, J.Z. Wu, W. Mao, Y. Gu, R. Zhao, J. Keppo, Y. Shan, M.Z. Shou, DynVideo-E: harnessing dynamic nerf for large-scale motion- and view-change human-centric video editing, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 7664–7674.
[24] C. Wang, M. Chai, M. He, D. Chen, J. Liao, CLIP-NeRF: text-and-image driven manipulation of neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 3835–3844.
[25] A. Haque, M. Tancik, A.A. Efros, A. Holynski, A. Kanazawa, Instruct-nerf2nerf: editing 3D scenes with instructions, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 19740–19750.
[26] H. Song, S. Choi, H. Do, C. Lee, T. Kim, Blending-NeRF: text-driven localized editing in neural radiance fields, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 14383–14393.
[27] L. Yu, W. Xiang, K. Han, Edit-DiffNeRF: Editing 3D neural radiance fields using 2D diffusion model, arXiv preprint arXiv:2306.09551, 2023.
[28] R. He, S. Huang, X. Nie, T. Hui, L. Liu, J. Dai, J. Han, G. Li, S. Liu, Customize your NeRF: adaptive source driven 3D scene editing via local-global iterative training, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 6966–6975.
[29] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al., Learning transferable visual models from natural language supervision, in: Proceedings of the International Conference on Machine Learning (ICML), 2021, pp. 8748–8763.
[30] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 10684–10695.
[31] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E.L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, et al., Photorealistic text-to-image diffusion models with deep language understanding, Adv. Neural Inf. Process. Syst. 35 (2022) 36479–36494.
[32] H. Cho, S.B. Kim, Improving diffusion-based image editing faithfulness via guidance and scheduling, arXiv preprint arXiv:2506.21045, 2025.
[33] Y.-H. Huang, Y. He, Y.-J. Yuan, Y.-K. Lai, L. Gao, StylizedNeRF: consistent 3D scene stylization as stylized nerf via 2D–3D mutual learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 18342–18352.
[34] J. Sun, X. Wang, Y. Zhang, X. Li, Q. Zhang, Y. Liu, J. Wang, FENeRF: face editing in neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 7672–7682.
[35] G. Lin, F.-L. Liu, S.-Y. Chen, K. Jiang, C. Li, Y. Lai, H. Fu, SketchFaceNeRF: sketch-based facial generation and editing in neural radiance fields, ACM Trans. Graph. 42 (4) (2023).
[36] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, Adv. Neural Inf. Process. Syst. 27 (2014).

[37] T. Brooks, A. Holynski, A.A. Efros, Instructpix2pix: learning to follow image editing instructions, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 18392–18402.
[38] S. Kobayashi, E. Matsumoto, V. Sitzmann, Decomposing NeRF for editing via feature field distillation, Adv. Neural Inf. Process. Syst. 35 (2022) 23311–23330.
[39] B. Poole, A. Jain, J.T. Barron, B. Mildenhall, DreamFusion: text-to-3D using 2D diffusion, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.
[40] X. Zhou, Y. He, F.R. Yu, J. Li, Y. Li, RePaint-NeRF: nerf editing via semantic masks and diffusion models, in: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, 2023, pp. 1813–1821.
[41] T. Wu, J.-M. Sun, Y.-K. Lai, L. Gao, De-nerf: decoupled neural radiance fields for view-consistent appearance editing and high-frequency environmental relighting, in: ACM SIGGRAPH 2023 Conference Proceedings, 2023, pp. 1–11.
[42] K. Kania, K.M. Yi, M. Kowalski, T. Trzciński, A. Tagliasacchi, CoNeRF: controllable neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 18623–18632.
[43] W. Cai, W. Liu, W. Li, Z. Zhao, F. Yin, X. Chen, L. Zhao, T. Chen, Instruct pix-to-3D: instructional 3D object generation from a single image, Neurocomputing 600 (2024) 128156.
[44] Y.-J. Yuan, Y.-T. Sun, Y.-K. Lai, Y. Ma, R. Jia, L. Gao, NeRF-Editing: geometry editing of neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 18353–18364.
[45] B. Yang, C. Bao, J. Zeng, H. Bao, Y. Zhang, Z. Cui, G. Zhang, NeuMesh: learning disentangled neural mesh-based implicit field for geometry and texture editing, in: Proceedings of the European Conference on Computer Vision (ECCV), 2022, pp. 597–614.
[46] K. Tertikas, D. Paschalidou, B. Pan, J.J. Park, M.A. Uy, I. Emiris, Y. Avrithis, L. Guibas, Generating part-aware editable 3D shapes without 3D supervision, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 4466–4478.
[47] Y. Zhang, Z. He, J. Xing, X. Yao, J. Jia, Ref-NPR: reference-based non-photorealistic radiance fields for controllable scene stylization, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 4242–4251.
[48] A. Mirzaei, T. Aumentado-Armstrong, K.G. Derpanis, J. Kelly, M.A. Brubaker, I. Gilitschenski, A. Levinshtein, SPIn-NeRF: multiview segmentation and perceptual inpainting with neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 20669–20679.
[49] Y. Hao, Y. Liu, Z. Wu, L. Han, Y. Chen, G. Chen, L. Chu, S. Tang, Z. Yu, Z. Chen, et al., EdgeFlow: achieving practical interactive segmentation with edge-guided flow, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 1551–1560.
[50] B. Galerne, J. Wang, L. Raad, J.-M. Morel, SGSST: scaling Gaussian splatting style transfer, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 26535–26544.
[51] Y. Mei, J. Xu, V. Patel, Regs: reference-based controllable scene stylization with Gaussian splatting, Adv. Neural Inf. Process. Syst. 37 (2024) 4035–4061.
[52] J.T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, P.P. Srinivasan, Mip-NeRF: a multiscale representation for anti-aliasing neural radiance fields, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 5855–5864.
[53] J.T. Barron, B. Mildenhall, D. Verbin, P.P. Srinivasan, P. Hedman, Mip-NeRF 360: unbounded anti-aliased neural radiance fields, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 5470–5479.
[54] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, 3D Gaussian splatting for real-time radiance field rendering, ACM Trans. Graph. 42 (2023) https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.
[55] D. Zhang, Y.-J. Yuan, Z. Chen, F.-L. Zhang, Z. He, S. Shan, L. Gao, Stylizedgs: controllable stylization for 3D Gaussian splatting, IEEE Trans. Pattern Anal. Mach. Intell. 47 (2025) 11961–11973, https://doi.org/10.1109/TPAMI.2025.3604010
[56] J.L. Schonberger, J.-M. Frahm, Structure-from-motion revisited, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 4104–4113.
[57] H. Aanæs, R.R. Jensen, G. Vogiatzis, E. Tola, A.B. Dahl, Large-scale data for multiple-view stereopsis, Int. J. Comput. Vis. 120 (2016) 153–168.
[58] B. Li, X. Qi, T. Lukasiewicz, P.H.S. Torr, ManiGAN: text-guided image manipulation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 7880–7889.
[59] R. Zhang, P. Isola, A.A. Efros, E. Shechtman, O. Wang, The unreasonable effectiveness of deep features as a perceptual metric, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 586–595.

Author biography



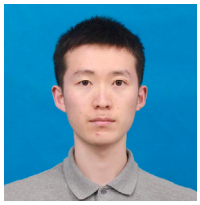
Haiyang Xu received the B.S. degree from the College of Geomatics, Xi'an University of Science and Technology, Xi'an, China, in 2021, and the M.S. degree from the School of Geomatics Science and Technology, Nanjing Tech University, Nanjing, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Statistics and Data Science, Nankai University, Tianjin, China. His research interests include machine learning and image processing.



Xia Xu received the B.S. and M.S. degrees from the School of Electrical Engineering, Yanshan University, China, in 2012 and 2015, respectively, and the Ph.D. degree from the School of Astronautics, Beihang University, China, in 2019. She is currently a Professor with the School of Computer Science and Technology, Tiangong University, Tianjin, China. Her research interests include hyperspectral unmixing, multiobjective optimization, and remote sensing image processing.



Yinghao Tan received the B.S. degree in Data Science and Big Data Technology from the School of Statistics and Data Science, Nankai University, Tianjin, China, in 2023. Currently, he is pursuing the M.S. degree in Applied Statistics at Nankai University. His research interests include machine learning and computer vision.



Ziye Zhang received the B.S. degree from the School of Statistics and Data Science, Nankai University, Tianjin, China, in 2024. He is currently working toward the M.S. degree in statistics at Nankai University. His research interests include physics-informed machine learning and interpretable neural networks.



Nanhe Chen received the B.S. degree in robotics engineering from Zhejiang University, Hangzhou, China, in 2023. He is currently pursuing the M.S. degree in control science and engineering with Zhejiang University, Hangzhou, China. His research interests include machine learning, motion planning, and multi-robot systems.



Bin Pan received the B.S. and Ph.D. degrees from the School of Astronautics, Beihang University, Beijing, China, in 2013 and 2019, respectively. He is currently a Professor with the School of Statistics and Data Science, Nankai University. He has authored or coauthored over 40 scientific articles. He serves as an Associate Editor for the Infrared Physics and Technology. His research interests include cross-domain remote sensing image processing, hyperspectral image classification, unmixing, and super-resolution. Codes for his articles are published online at https://github.com/Lab-PANbin/.



Zhenwei Shi received the Ph.D. degree in mathematics from the Dalian University of Technology, Dalian, China, in 2005. He was a Post-Doctoral Researcher with the Department of Automation, Tsinghua University, Beijing, China, from 2005 to 2007. He was a Visiting Scholar with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL, USA, from 2013 to 2014. He is currently a Professor and the Dean of the Department of Aerospace Intelligent Science and Technology, School of Astronautics, Beihang University, Beijing, China. He has authored or co-authored over 300 scientific articles in refereed journals and proceedings. His research interests include remote sensing image processing and analysis, computer vision, pattern recognition, and machine learning., Prof. Shi serves as an Editor for IEEE Transactions on Geoscience and Remote Sensing, Pattern Recognition, ISPRS Journal of Photogrammetry and Remote Sensing, and Infrared Physics and Technology. His personal website is https://levir.buaa.edu.cn/.